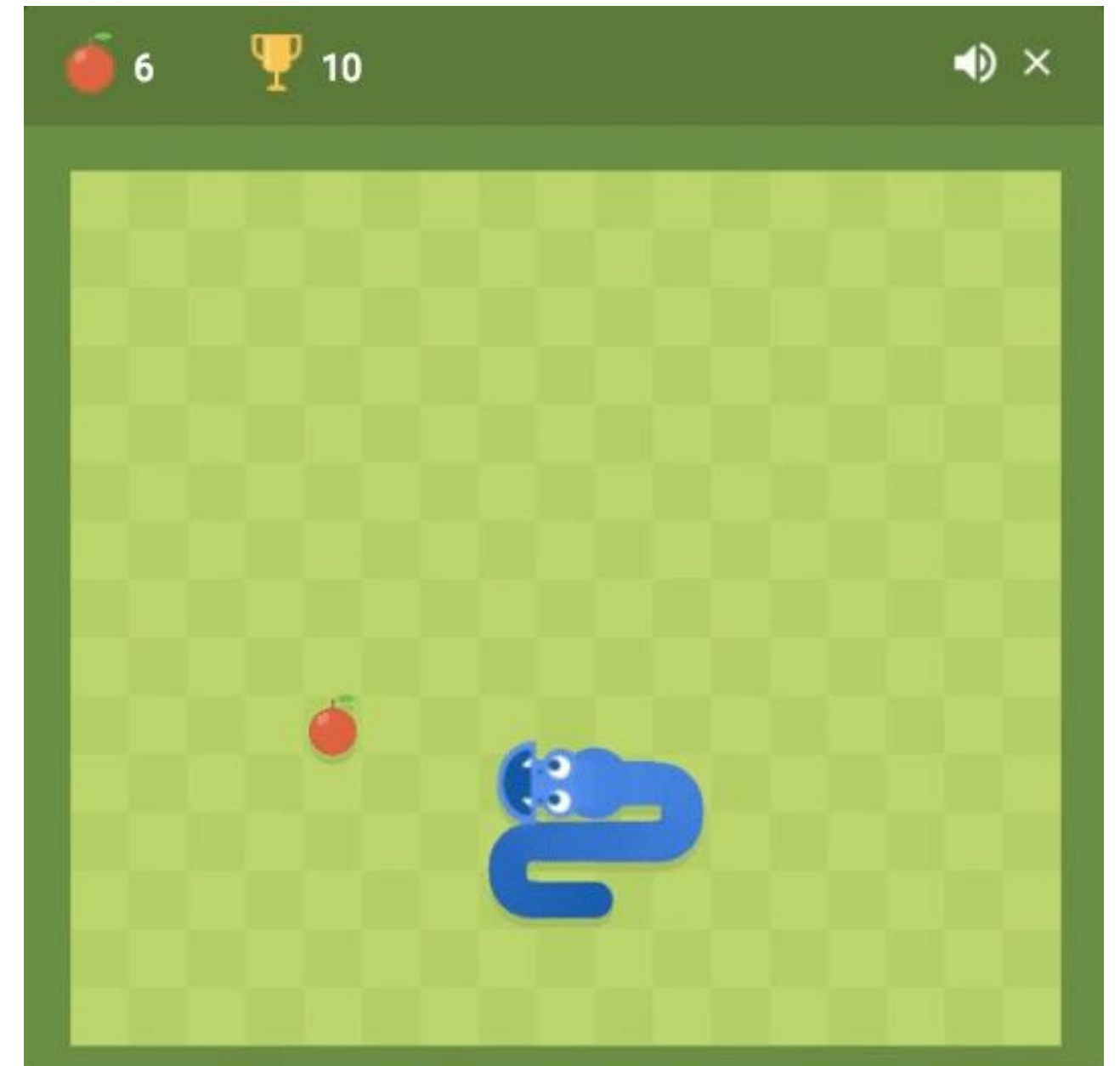


INTRODUCTION TO THE SNAKE GAME

THIS SESSION WILL COVER HOW TO CREATE A SIMPLE YET ADDICTIVE SNAKE GAME USING PYTHON'S TURTLE GRAPHICS LIBRARY WHILE EMPHASIZING FUNDAMENTAL PROGRAMMING CONCEPTS.

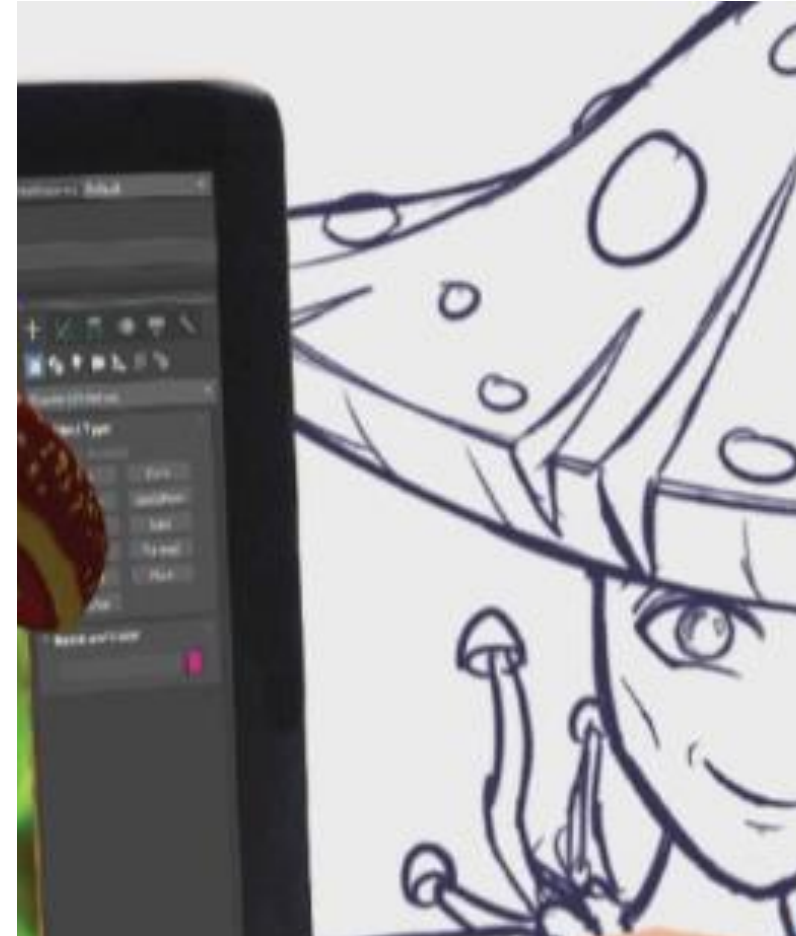


GAME OVERVIEW

The Snake game is a classic arcade experience where players control a growing snake, aiming to score points by eating food. The challenge lies in avoiding collisions with walls and the snake's own body, making it both engaging and addictive.



```
File Edit Selection View Go Run ... Search
prog 14.py X
C: > Users > harsh > OneDrive > Desktop > prog 14.py > ...
1 import turtle
2 import time
3 import random
4
5 # Set up the screen
6 win = turtle.Screen()
7 win.title("Snake Game")
8 win.bgcolor("pink")
9 win.setup(width=600, height=600)
10 win.tracer(0) # Turns off auto screen updates
11
12 # Snake Head
13 head = turtle.Turtle()
14 head.shape("square")
15 head.color("white")
16 head.penup() # iske bina snake k piche line aaegi
17 head.goto(0, 0)
18 head.direction = "stop"
19
20 # Snake Food
21 food = turtle.Turtle()
22 food.speed(0)
23 food.shape("circle")
24 food.color("brown")
25 food.penup()
26 food.goto(0, 100)
27
28 # Snake Body Segments
29 segments = []
30
31 # Score
32 score = 0
```



TURTLE GRAPHICS



Turtle graphics is a popular way to introduce programming to beginners through simple visual feedback. It allows the drawing of shapes and images on the screen by controlling a turtle, making it easy to create engaging games like Snake.

MOVEMENT AND COLLISION LOGIC

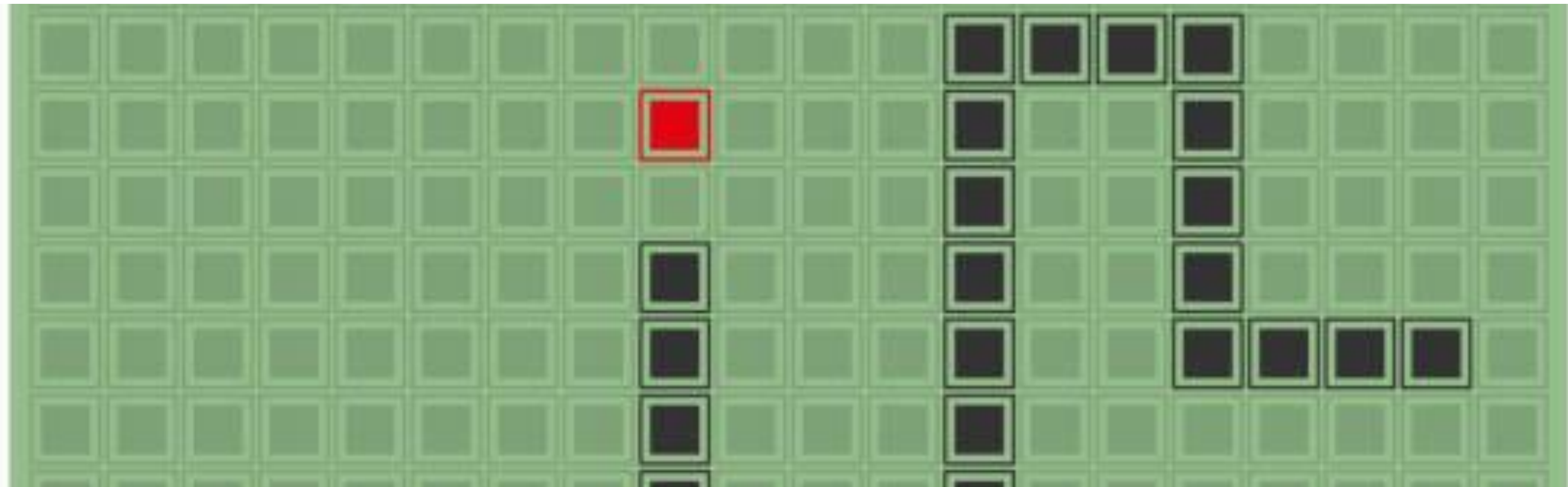
THE CORE OF THE SNAKE GAMEPLAY RELIES ON BASIC MOVEMENT AND COLLISION DETECTION. THE PLAYER NAVIGATES THE SNAKE BY CHANGING ITS DIRECTION, WITH LOGIC IMPLEMENTED TO DETECT COLLISIONS WITH GAME BOUNDARIES AND ITSELF, ENHANCING THE GAME'S CHALLENGE.

KEYBOARD BINDINGS

KEYBOARD BINDINGS ARE ESSENTIAL FOR PLAYER CONTROL WITHIN THE SNAKE GAME. UTILIZING FUNCTIONS LIKE `'WIN.LISTEN()'` AND `'WIN.ONKEY()'`, PLAYERS CAN CONTROL THE SNAKE'S DIRECTION, CREATING AN INTERACTIVE GAMING EXPERIENCE THAT RELIES ON RESPONSIVE INPUT.

GAME OBJECTIVE

THE OBJECTIVE OF THE SNAKE GAME CENTERS ON SCORING POINTS WHILE NAVIGATING WITHOUT COLLISIONS, OFFERING AN ENGAGING CHALLENGE FOR PLAYERS.



COLLISIONS

1. FOOD COLLISION LOGIC

```
# Check for collision with food
if head.distance(food) < 20:
    # Move food to random spot
    x = random.randint(-290, 290)
    y = random.randint(-290, 290)
    food.goto(x, y)

    # Add segment to snake
    new_segment = turtle.Turtle()
    new_segment.speed(0)
    new_segment.shape("square")
    new_segment.color("sky blue")
    new_segment.penup()
    segments.append(new_segment)

    # Increase score
    score += 10
    score_display.clear()
    score_display.write(f"Score: {score}", align="center", font=("Courier", 24, "underline"))
```

2. SELF COLLISIONS

```
# Check for collision with self
for segment in segments:
    if segment.distance(head) < 20:
        time.sleep(1)
        head.goto(0, 0)
        head.direction = "stop"

        # Hide segments
        for segment in segments:
            segment.goto(1000, 1000)
        segments.clear()

    score = 0
    score_display.clear()
    score_display.write(f"Score: {score}", align="center", font=("Courier", 24, "underline"))
```

```
# Check for collision with border
if head.xcor() > 290 or head.xcor() < -290 or head.ycor() > 290 or head.ycor() < -290:
    time.sleep(1)
    head.goto(0, 0)
    head.direction = "stop"

    # Hide segments
    for segment in segments:
        segment.goto(1000, 1000)
    segments.clear()

    score = 0
    score_display.clear()
    score_display.write(f"Score: {score}", align="center", font=("Courier", 24, "underline"))
```

3. WALL COLLISIONS

CODE WALKTHROUGH

```
snake game.py - C:/Users/harsh/AppData/Local/Programs/Python/Python313/snake game.py (3.13.0)
File Edit Format Run Options Window Help

import turtle
import time
import random

# Set up the screen
win = turtle.Screen()
win.title("Snake Game")
win.bgcolor("pink")
win.setup(width=600, height=600)
win.tracer(0) # Turns off auto screen updates

# Snake Head
head = turtle.Turtle()
head.shape("square")
head.color("white")
head.penup()
head.goto(0, 0)
head.direction = "stop"

# Snake Food
food = turtle.Turtle()
food.speed(0)
food.shape("circle")
food.color("brown")
food.penup()
food.goto(0, 100)

# Snake Body Segments
segments = []

# Score
score = 0

# Display Score
score_display = turtle.Turtle()
score_display.speed(0)
score_display.color("white")
score_display.penup()
score_display.hideturtle()
score_display.goto(0, 260)
score_display.write("Score: 0", align="center", font=("Courier", 24, "underline"))

# Functions
def move():
    if head.direction == "up":
        y = head.ycor()
        head.sety(y + 20)
    if head.direction == "down":
        y = head.ycor()
        head.sety(y - 20)
```

Ln: 152 Col: 0

```
        head.sety(y - 20)
    if head.direction == "left":
        x = head.xcor()
        head.setx(x - 20)
    if head.direction == "right":
        x = head.xcor()
        head.setx(x + 20)

def go_up():
    if head.direction != "down":
        head.direction = "up"

def go_down():
    if head.direction != "up":
        head.direction = "down"

def go_left():
    if head.direction != "right":
        head.direction = "left"

def go_right():
    if head.direction != "left":
        head.direction = "right"

# Keyboard bindings
win.listen()
win.onkey(go_up, "w")
win.onkey(go_down, "s")
win.onkey(go_left, "a")
win.onkey(go_right, "d")

# Main game loop
while True:
    win.update()

    # Check for collision with border
    if head.xcor() > 290 or head.xcor() < -290 or head.ycor() > 290 or head.ycor() < -290:
        time.sleep(1)
        head.goto(0, 0)
        head.direction = "stop"

        # Hide segments
        for segment in segments:
            segment.goto(1000, 1000)
        segments.clear()

        score = 0
        score_display.clear()
        score_display.write(f"Score: {score}", align="center", font=("Courier", 24, "underline"))
```

```
# Check for collision with food
if head.distance(food) < 20:
    # Move food to random spot
    x = random.randint(-290, 290)
    y = random.randint(-290, 290)
    food.goto(x, y)

    # Add segment to snake
    new_segment = turtle.Turtle()
    new_segment.speed(0)
    new_segment.shape("square")
    new_segment.color("sky blue")
    new_segment.penup()
    segments.append(new_segment)

    # Increase score
    score += 10
    score_display.clear()
    score_display.write(f"Score: {score}", align="center", font=("Courier", 24, "underline"))

# Move the end segments in reverse order
for index in range(len(segments) - 1, 0, -1):
    x = segments[index - 1].xcor()
    y = segments[index - 1].ycor()
    segments[index].goto(x, y)

# Move segment 0 to where the head is
if len(segments) > 0:
    x = head.xcor()
    y = head.ycor()
    segments[0].goto(x, y)

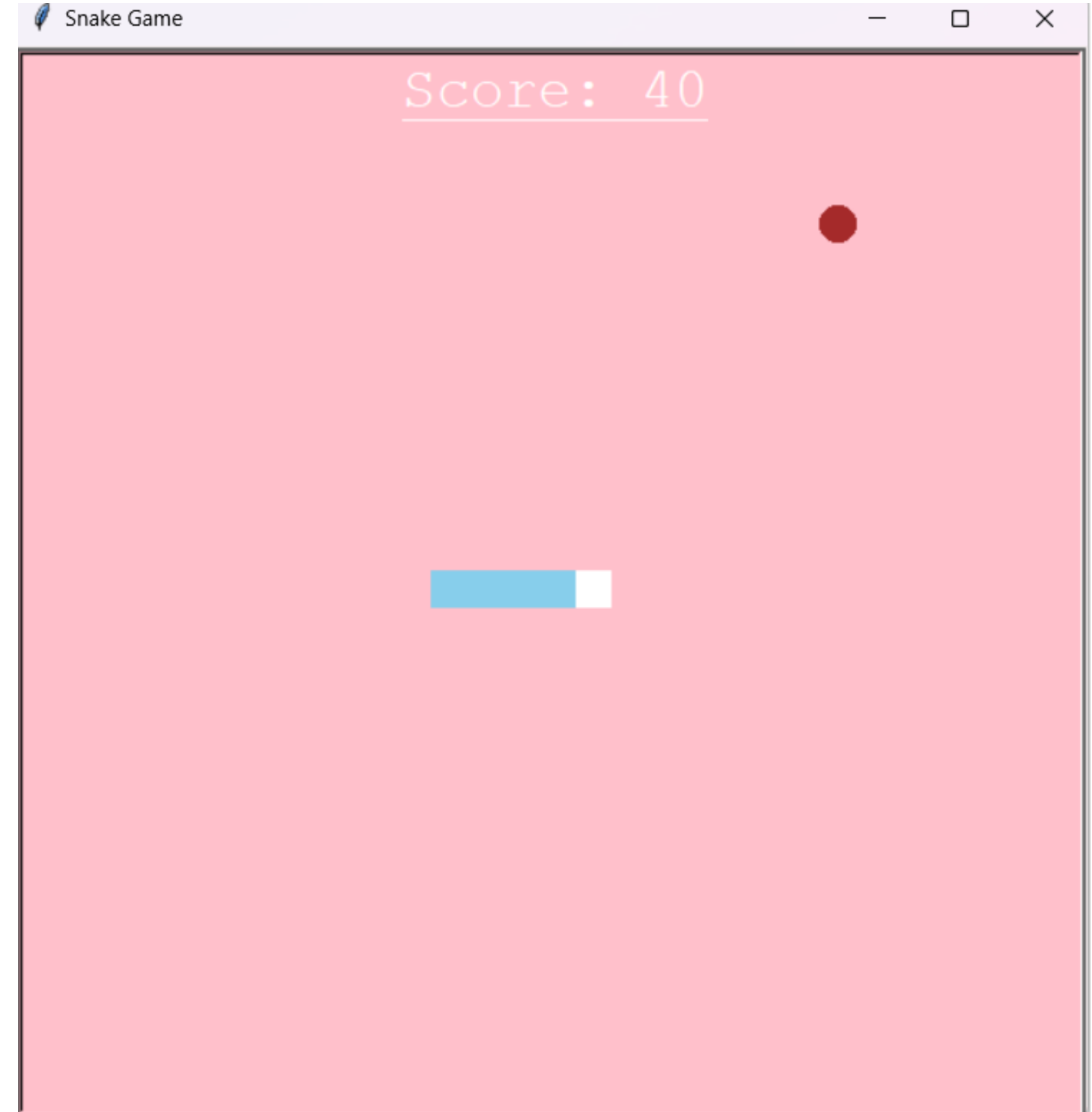
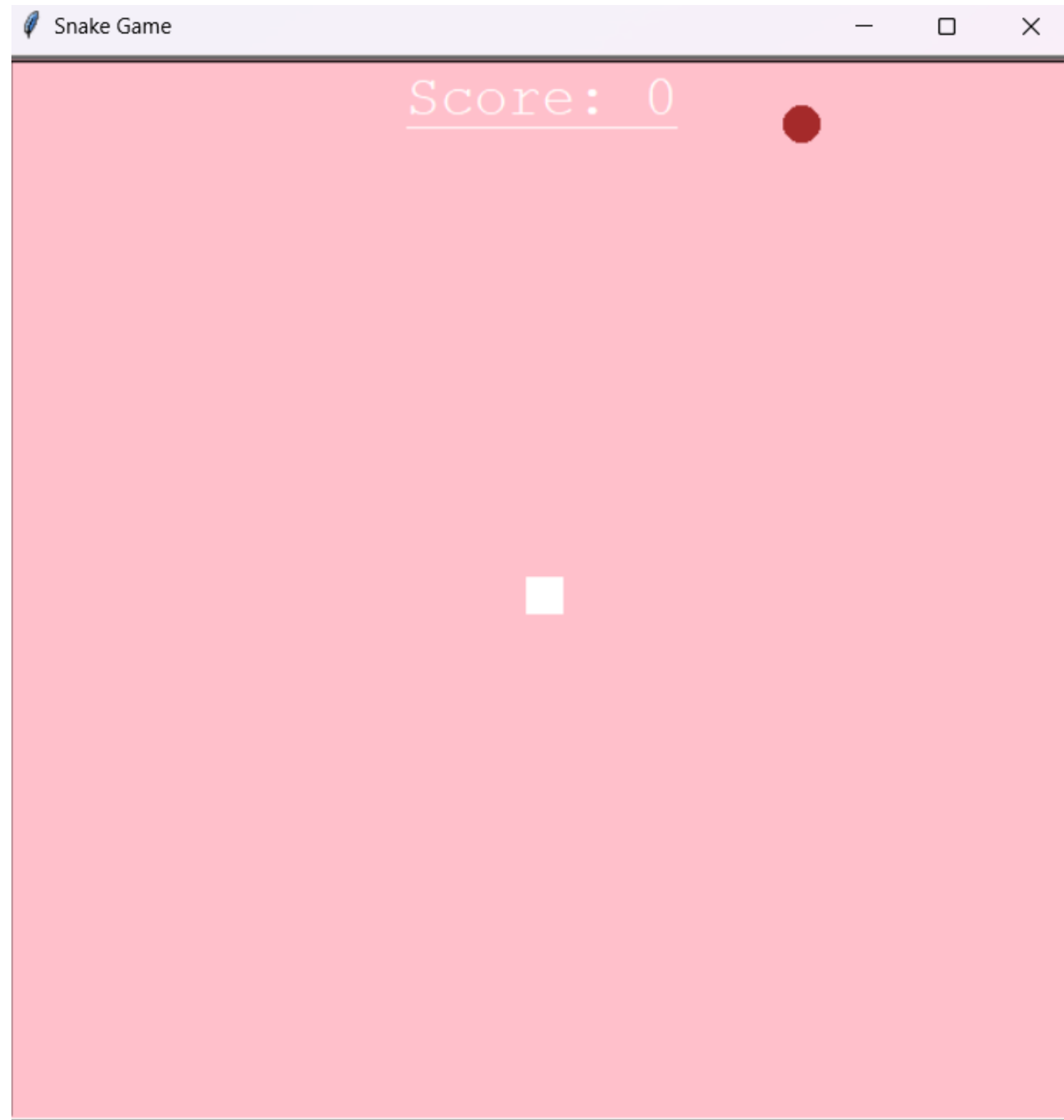
move()

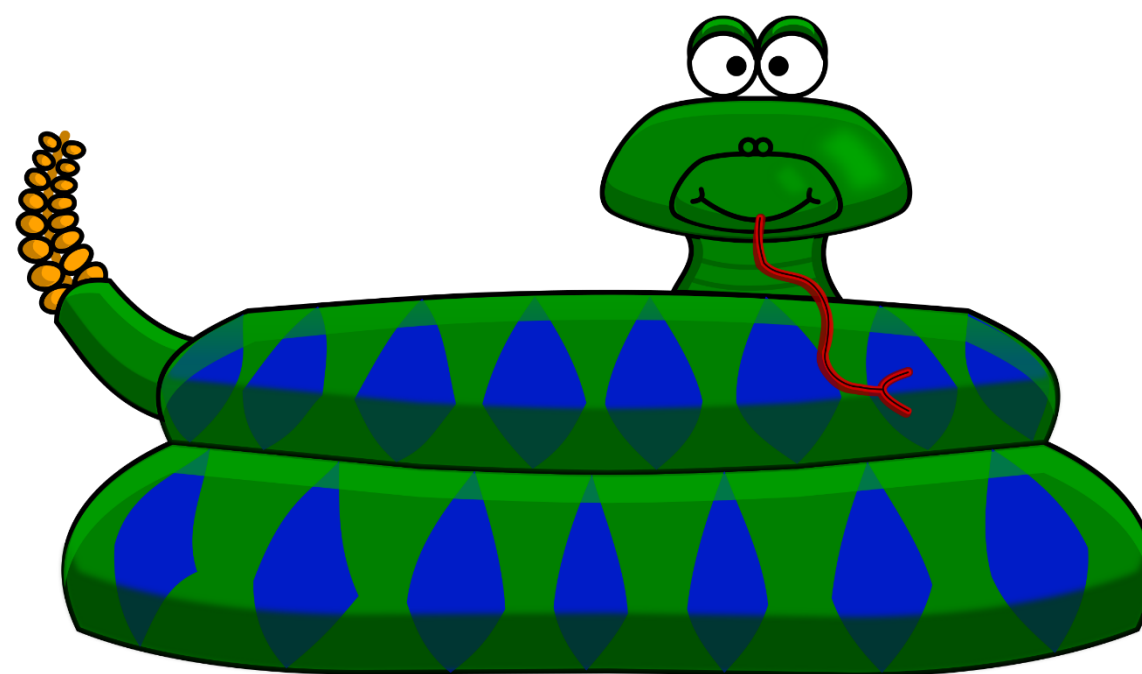
# Check for collision with self
for segment in segments:
    if segment.distance(head) < 20:
        time.sleep(1)
        head.goto(0, 0)
        head.direction = "stop"

        # Hide segments
        for segment in segments:
            segment.goto(1000, 1000)
        segments.clear()

        score = 0
        score_display.clear()
        score_display.write(f"Score: {score}", align="center", font=("Courier", 24, "underline"))
time.sleep(0.1)
```

DEMONSTRATION AND INTERACTION





**THANK
YOU!!**